CSE470: Software Engineering

Software Requirements Specification (SRS)

SHARE (Systematic Hazard Assessment & Relief Engine)

Prepared By

Student ID	Name
24301374	Shadeed Aarshan
24301362	Nuzhat Tabassum Oishee
24341019	Mehbuba Uddin
22201593	Jarif Mustafiz

Project Information

Section	11
Section Faculty	Nazmus Sakib Touhid(NAST)
GitHub Repository Link	https://github.com/oishee296/CSE470_Group05
Tech Stack Used	*Frontend: React* *Backend Layer: PHP, LaravelWeb Engine* *Database: MySQL*

1. Introduction

1.1 Purpose

The purpose of this document is to specify the complete functional and non-functional requirements for the Systematic Hazard Assessment & Relief Engine (SHARE). SHARE is a comprehensive web application engineered to orchestrate emergency disaster responses, streamline donor contributions, automate centralized inventory tracking, and manage field volunteer deployments during catastrophic environmental crises.

1.2 Scope

SHARE acts as an operational node connecting victims, humanitarian donors, logistical warehouse managers, and emergency field coordinators. The system handles victim SOS logging, resource allocations, smart shelter tracking, automated aid kit assemblies, and transparent public tracking. Explicitly out of scope are real-time GPS coordinate satellite tracking, automated payment gateway clearing systems.

2. Overall Description

2.1 Product Perspective

SHARE operates as a centralized web platform that replaces uncoordinated manual communication sheets used by separate NGOs. It acts as an integrated system communicating with local administrative regional databases to pull regional census metrics while operating independently to manage material assets, field personnel, and citizen feedback loops.

2.2 Product High-Level Summary

The system provides immediate emergency relief allocation to individuals affected by ecological disasters. It balances incoming supply chains from civil donors against the live demand calculated from victim SOS metrics, utilizing structured algorithmic routing to minimize human distribution delays.

2.3 Users and Characteristics

Affected Citizen (Victim): Displaced or vulnerable individuals needing immediate food, shelter, or rescue assistance. Low to moderate technical literacy.

Humanitarian Donor: Individuals or corporate entities providing capital or material assets. Requires transaction logs and tax summaries.

Logistics Manager: Personnel stationed at local supply warehouses who manage inventory entries, goods packing, and transport fleets.

Command Center Admin: Advanced system coordinators who register active disasters, vet field personnel, and analyze fulfillment metrics.

2.4 Operating Environment

Client Side: Modern web browsers (Google Chrome, Mozilla Firefox, Safari)

Server Side: PHP, MySQL.

2.5 Design and Implementation Constraints

This project adheres strictly to the CSE470 architecture guidelines:

(a) Primary Tech Stack: PHP, Bootstrap, MySQL.

(b) MVC Framework Structure: Structured via the Laravel MVC Framework. Models are isolated in app/Models/, Controllers live inside app/Http/Controllers/, and UI files use the Blade engine in resources/views/.

(c) AI Features: No external Python/FastAPI tool dependencies are deployed for core features to maintain native architectural purity.

(d) Exceptions: None requested.

2.6 Assumptions and Dependencies

Assumes persistent internet availability within regional local deployment sectors and uninterrupted relational uptime for the central database instance during peak transaction workflows.

3. System Architecture

3.1 Architectural Overview

[Insert or describe an MVC architecture diagram showing how the Model, View, and Controller layers are organized in your project, and how they communicate.]

3.2 Technology Stack

Frontend Layer: React

Backend Layer: PHP, Laravel Web Engine (Eloquent ORM Routing Components).

Database Layer: MySQL Relational Management Database System.

Hosting/Deployment: Local host environment configuration via Git repository pipelines.

3.3 Class Diagram

[Draw the Class Diagram of your project showing all the classes and relationships necessary to implement your system]

4. Requirements and Features

List every requirement and its associated feature(s) below. Per the Project Outline (Section 3.2.1), all CRUD operations on the same entity by the same actor using the same logic count as ONE feature — do not split them. A minimum of 20 features is required.

[Replace the sample row with your own rows. Add as many rows as needed. Each Feature ID should be unique (F1, F2, F3…) and referenced consistently across this document and your Git commit history.]

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R1	Affected Citizens can manage distress signals.	SOS Aid Request Workflow: Victims log, view, edit, or cancel active rescue requests specifying category (Medical, Food, Rescue) and location metrics.	Affected Citizen	M: ReliefRequest V: requests/create.blade, requests/index.blade C: ReliefRequestController		
R2	Victims can record household size details.	Family Vulnerability Tracking: Setting household size profiles and flags (infants, elderly, injured counts) to dynamically generate regional severity weights.	Affected Citizen	M: FamilyProfile V: profile/family.blade C: FamilyProfileController	…	…
R3	Displaced persons can request safe accommodation.	Shelter Booking & Check-in: Real-time viewing of active community rescue camps to register and secure	Affected Citizen	…	…	…

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
		sleeping capacity slots for household groups.				
R4	Citizens can access localized evacuation directions.	Evacuation Route Tracker: Viewing active safe sector zones and downloading static physical step-by-step route directions assigned to their ward	Affected Citizen			
R5	Recipients can verify aid package arrivals.	Victim Feedback & Delivery Confirmation: Submitting formal confirmations when resource batches arrive or logging reports detailing missing supplies	Affected Citizen			
R6	Public citizens can log material resources.	Supply Donation Logistics: Donors register drop-off asset packages (blankets, clean water, medical packs) into a tracking lane and print receipts.	Humanitarian Donor			
R7	Philanthropists can fund localized micro-campaigns.	Financial Pledge & Micro-Funding: Allocating monetary credits directly to specific disaster areas or camp instances with live progression bars	Humanitarian Donor			
R8	Institutional entities can track aggregate aid history.	Corporate Donor Dashboard: Tools allowing bulk enterprise groups to manage item allocations and export certified annual tax-exemption sheets	Humanitarian Donor			
R9	Donors can fulfill urgent warehouse requests.	Item Wishlist Matching: Browsing specific, urgent supply shortages requested by warehouse hubs and mapping item donations directly to them	Humanitarian Donor			

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R10	Public users can initialize custom resource drives.	Campaign Creator: Organizing external community-led collection initiatives for newly declared hazard zones to gather localized inventory	Humanitarian Donor			
R11	Storage staff can control regional resource supplies.	Warehouse Hub Management: Complete operational tracking of resource volume capacities, asset incoming logs, and storage locker assignments	Logistics Manager			
R12	Logisticians can compile combined asset packages.	Relief Kit Assembly Workflow: Packaging multiple distinct inventory lines (e.g., 2 blankets, 5 water bottles) into unified "Standard Aid Bundles".	Logistics Manager			
R13	Operational staff can oversee supply vehicles.	Dispatch & Transport Logistics: Assigning localized transport vehicles or dispatch drivers to haul assembled aid batches to frontline centers.	Logistics Manager	M: TransportLog, Vehicle V: logistics/dispatch.blade C: LogisticsController		
R14	Workers can manage expiring emergency goods.	Perishables & Medicine Expiration Alert: Tracking batch validation markers on foodstuffs or clinical components to prioritize aging lots.	Logistics Manager	M: Inventory V: warehouse/alerts.blade C: ExpirationController		
R15	Purchasing staff can manage raw supplier streams.	Supplier Procurement Engine: Managing external commercial vendors to automatically re-order abulk dry items when inventory dips below minimum margins.	Logistics Manager	M: Vendor, PurchaseOrder V: procurement/orders.blade		

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
				C: ProcurementController		
R16	System Admins can track macro crisis entries.	Disaster Declaration & Zoning: Defining a new hazard event profile, mapping impacted regional sub-districts, and configuring priority tiers.	Command Center Admin	M: Disaster, Zone V: admin/disaster_declare.blade C: DisasterController		
R17	Personnel staff can handle regional staff tracking.	Volunteer Deployment Workflow: Screening incoming medical or rescue helper registrations, validating certifications, and mapping profiles to active units.	Command Center Admin	M: VolunteerProfile, DutyAssignment V: volunteers/vetting.blade, volunteers/roster.blade C: VolunteerController		
R18	Coordinators can compare net demand against stock.	Real-Time Need Assessment Grid: An analytical matrix matching cumulative citizen requested metrics directly against active warehouse inventory holdings.	Command Center Admin	M: ReliefRequest, Inventory V: admin/assessment_grid.blade C: AssessmentController		
R19	Administrators can force priority adjustments.	Incident Escalation Engine: Automatically or manually upgrading unresolved rescue ticket urgency profiles using chronological baseline checks.	Command Center Admin	M: ReliefRequest V: admin/escalations.blade C: EscalationController		

Req. ID	Requirement	Feature Description	Actor	MVC Components Touched	Sprint Completed	Status
R20	Oversight officers can audit relief item pipelines.	Audit & Transparency Logs: System-wide immutable reporting modules tracing exact end-to-end routing pathways for every financial or material item.	Command Center Admin	M: AuditTrail, Donation V: admin/audit_report.blade C: AuditController		

4.1 Feature Count Summary

Minimum features required (per Project Outline, Section 7)	20
Total features listed in Section 4 above	20
Total features fully completed (“Done” per Section 7.2 of the Project Outline)	

5. External Interface Requirements (Optional)

Interface Type	Description
User Interface	Clean and intuitive.Pages should load within a few seconds.
Hardware Interface	No additional hardware required.
Software Interface (APIs, third-party services)	
Communication Interface	

6. Non-Functional Requirements (Optional)

Category	Requirement
Performance	Interface data displays must process queries within less than 2.0 seconds during concurrent user transactions.
Security	Multi-tier role permissions enforced strictly via session tokens to isolate victim profiles from administrative operational dashboards
Usability	Forms optimized for quick data entry with clear text fields and large action confirmation buttons

Category	Requirement
Reliability / Availability	
Scalability	

7. Sprint History and Review Ceremony Log

Record the goals and attainment shared during each weekly sprint review ceremony, per Section 7.1 of the Project Outline.

Sprint	Goals Set (Start of Week)	Attainment Reported (End of Week)	Features Completed This Sprint
Sprint 1			
Sprint 2			
Sprint 3			
Sprint 4			

7.1 Deviations and Lessons Learned

[Briefly explain any sprint where attainment did not match the planned goals, why it happened, and what was adjusted for the following sprint.]

8. Testing Summary

[Briefly describe how each major feature was tested (manual testing steps, sample test cases, or automated tests). A short table or list is acceptable.]

9. AI Tool Usage Disclosure

Per the Project Outline's Academic Integrity & AI Usage Policy (Section 6), disclose every instance where a generative AI tool was used during development. Leave the table empty (do not delete it) if no AI tool was used.

Feature / Task	AI Tool Used	How It Was Used (be specific)	I Can Fully Explain This Code (Yes/No)

[By submitting this document, you confirm that you are able to personally explain, modify, and debug every piece of code listed above during your viva.]

10. Individual Contribution Summary

[Since each student develops their project individually, summarize your own work: which features you implemented, approximate time spent per sprint, and any challenges you solved independently. If you collaborated with your 4-student group on non-code matters (e.g. idea discussion), state that clearly and distinguish it from your individual implementation work.]

11. Pre-Submission Checklist

Checklist Item	Confirmed (Yes/No)
The application implements the MVC architecture	YES
At least 20 features are listed and each is correctly counted per the granularity rule	YES
No forbidden framework (e.g. Django, Flask as primary backend) was used	YES
All four sprints' goals and attainment are recorded in Section 7	
The GitHub repository is up to date and the section faculty is added as a collaborator	YES
The AI Usage Disclosure table (Section 10) is complete and honest	
I can personally explain and defend every feature listed in this document during the viva	YES